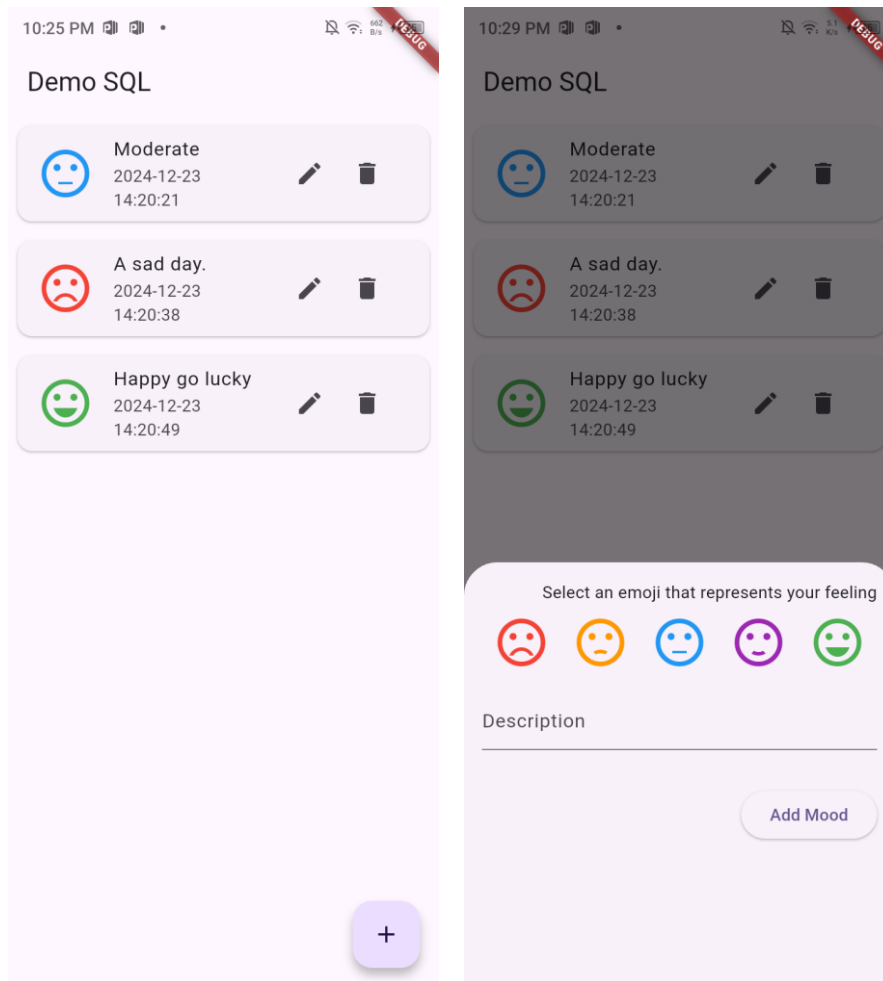


Practical 9: SQLite

1. Create a new Flutter app using the Application template. It is a mood journal app allowing users to perform emotion tracking.



2. In the Terminal, add the following dependencies:

```
flutter pub add sqflite
flutter pub add uuid
flutter pub add path_provider
```

3. Insert a new dart file named mood_model.dart. This is a model class to perform CRUD operations of mood records.

```
class MoodModel{
  final int id;
  final int scale;
  final String description;
  final String createdOn;
```

```

MoodModel({
  required this.id,
  required this.scale,
  required this.description,
  required this.createdOn,
});

factory MoodModel.fromJson(Map<String, dynamic>data) => MoodModel(
  id: data['id'],
  scale: data['scale'],
  description: data['description'],
  createdOn: data['createdOn'],
);

Map<String, dynamic> toMap() => {
  'id': id,
  'scale': scale,
  'description': description,
  'createdOn': createdOn,
};
}

```

4. Insert a new dart file named database_service.dart. This class performs all the database operations.

```

import 'dart:developer';
import 'package:demo_sqlite/mood_model.dart';
import 'package:path_provider/path_provider.dart';
import 'package:sqflite/sqflite.dart';

class DatabaseService{
  static final DatabaseService _databaseService = DatabaseService._internal();
  factory DatabaseService() => _databaseService;
  DatabaseService._internal();
  static Database? _database;

  //Get an instance of database
  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await initDatabase();
    return _database!;
  }

  //Initialize a database
  Future<Database> initDatabase() async {
    final getDirectory = await getApplicationDocumentsDirectory();
    String path = '${getDirectory.path}/moods.db';
    log(path);
    return await openDatabase(path, onCreate: _onCreate, version: 1);
  }
}

```

```
//Create an instance of database
void _onCreate(Database db, int version) async {
  await db.execute(
    'CREATE TABLE Moods('
      'id INTEGER PRIMARY KEY AUTOINCREMENT, '
      'scale INTEGER, '
      'description TEXT, '
      'createdOn DATETIME DEFAULT CURRENT_TIMESTAMP)');
  log('TABLE CREATED');
}

Future<List<MoodModel>> getMood() async {
  final db = await _databaseService.database;
  var data = await db.query('Moods');
  List<MoodModel> moods =
  List.generate(data.length, (index) => MoodModel.fromJson(data[index]));
  print(moods.length);
  return moods;
}

Future<void> insertMood(MoodModel mood) async {
  final db = await _databaseService.database;
  var data = await db.rawInsert(
    'INSERT INTO Moods(scale, description) VALUES(?,?)',
    [mood.scale, mood.description]);
  log('inserted $data');
}

Future<void> editMood(MoodModel mood) async {
  final db = await _databaseService.database;
  var data = await db.update('Moods', mood.toMap(), where: 'id=?', whereArgs:
[mood.id]);
  log('updated $data');
}

Future<void> deleteMood(int id) async {
  final db = await _databaseService.database;
  var data = await db.delete('Moods', where: 'id = ?', whereArgs: [id]);
  log('deleted $data');
}
}
```

5. In the **main.dart** file, within the Stateful widget class:

```
//TODO - Declare instance of database service and local variables
final dbService = DatabaseService();
final descriptionController = TextEditingController();
int _scale = 3;
```

6. In the **main.dart** file, within the Stateful widget class, insert a function named `showBottomSheet`. This function will prompt a Modal Bottom Sheet, which users can insert their mood journals.

```
//TODO - Insert showBottomSheet

void showBottomSheet(String functionTitle, Function()? onPressed) {
  showModalBottomSheet(
    context: context,
    elevation: 5,
    isScrollControlled: true,
    builder: (_) => Container(
      padding: EdgeInsets.only(
        top: 15, left: 15, right: 15,
        bottom: MediaQuery.of(context).viewInsets.bottom + 120,
      ),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.end,
        children: [
          Text('Select an emoji that represents your feeling',),
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceAround,
            children: [
              IconButton(
                icon: Icon(Icons.sentiment_very_dissatisfied),
                iconSize: 48,
                color: Colors.red,
                onPressed: () {
                  _scale = 1;
                },
              ),
              IconButton(
                icon: Icon(Icons.sentiment_dissatisfied),
                iconSize: 48,
                color: Colors.orange,
                onPressed: () {
                  _scale = 2;
                },
              ),
              IconButton(
                icon: Icon(Icons.sentiment_neutral),
                iconSize: 48,
                color: Colors.blue,
                onPressed: () {
                  _scale = 3;
                },
              ),
              IconButton(
                icon: Icon(Icons.sentiment_satisfied),
                iconSize: 48,
                color: Colors.purple,
                onPressed: () {
                  _scale = 4;
                },
              ),
              IconButton(
                icon: Icon(Icons.sentiment_very_satisfied),
                iconSize: 48,
                color: Colors.green,
                onPressed: () {
                  _scale = 5;
                },
              ),
            ],
          ),
        ],
      ),
    ),
  );
}
```

```

    ),
    const SizedBox(
      height: 10,
    ),
    TextField(
      controller: descriptionController,
      keyboardType: TextInputType.text,
      decoration: const InputDecoration(
        hintText: 'Description'),
    ),
    const SizedBox(
      height: 10,
    ),
    const SizedBox(
      height: 20,
    ),
    ElevatedButton(
      onPressed: onPressed,
      child: Text(functionTitle),
    )
  ],
),
));
}

```

7. In the main.dart file, within the Stateful widget class, insert 3 functions to add, edit, and delete emotion journals:

```

void addMood() {
  showBottomSheet('Add Mood', () async {
    var mood = MoodModel(
      id: 0,
      scale: _scale,
      description: descriptionController.text,
      createdOn: '',
    );

    dbService.insertMood(mood);
    //Update UI
    setState(() {});
    //Clear input
    descriptionController.clear();
    //Close bottom sheet
    Navigator.of(context).pop();
  });
}

void editMood(MoodModel mood) {
  descriptionController.text = mood.description;

  showBottomSheet('Update Mood', () async {
    var updatedMood = MoodModel(
      id: mood.id,
      scale: _scale,
      description: descriptionController.text,
      createdOn: mood.createdOn,
    );

    dbService.editMood(updatedMood);
    descriptionController.clear();
  });
}

```

```

        setState(() {});
        Navigator.of(context).pop();
    });
}

void deleteMood(int id) {
    dbService.deleteMood(id);
    setState(() {});
}

```

8. Modify the build function of the Stateful widget:

```

//TODO - Modify the build function
@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
            title: const Text('Demo SQL'),
        ),
        body: FutureBuilder<List<MoodModel>>(
            future: dbService.getMood(),
            builder: (context, snapshot) {
                if (snapshot.connectionState == ConnectionState.waiting) {
                    return const Center(child: CircularProgressIndicator());
                }
                if (snapshot.hasData) {
                    if (snapshot.data!.isEmpty) {
                        return const Center(
                            child: Text('No record found'),
                        );
                    }
                    return ListView.builder(
                        itemCount: snapshot.data!.length,
                        itemBuilder: (context, index) => Card(
                            margin: const EdgeInsets.all(8),
                            child: ListTile(
                                leading: emojiIcon(snapshot.data![index].scale),
                                title: Text(snapshot.data![index].description),
                                subtitle: Text(snapshot.data![index].createdOn),
                                trailing: SizedBox(
                                    width: 100,
                                    child: Row(
                                        children: [
                                            IconButton(
                                                icon: const Icon(Icons.edit),
                                                onPressed: () =>
                                                    editMood(snapshot.data![index]),
                                            ),
                                            IconButton(
                                                icon: const Icon(Icons.delete),
                                                onPressed: () =>
                                                    deleteMood(snapshot.data![index].id),
                                            ),
                                        ],
                                    ),
                                ),
                            ),
                        ),
                    ),
                ),
            ),
        ),
    );
}

return const Center(
    child: Text('No record found'),
);
}),

```

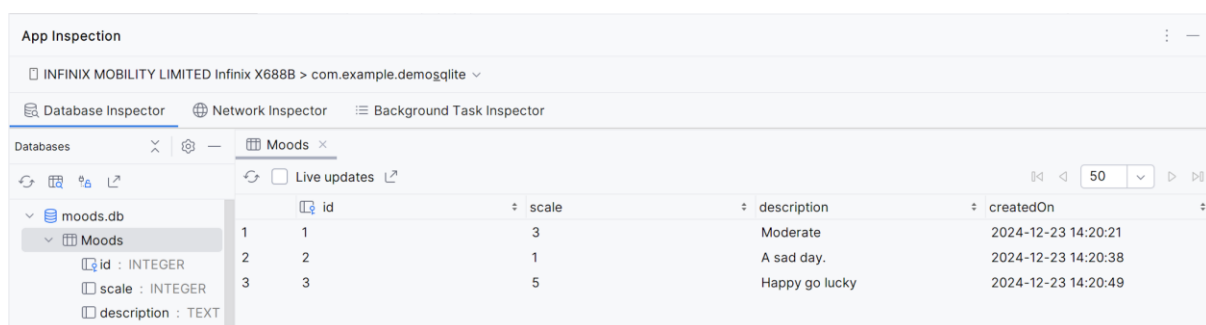
```
floatingActionButton: FloatingActionButton(
  child: const Icon(Icons.add),
  onPressed: () => addMood(),
),
);
}
```

9. Within the main.dart class, insert a function named emojiIcon:

```
Icon emojiIcon(int scale) {
  switch (scale) {
    case 1:
      return Icon(
        Icons.sentiment_very_dissatisfied,
        size: 48,
        color: Colors.red,
      );
    case 2:
      return Icon(
        Icons.sentiment_dissatisfied,
        size: 48,
        color: Colors.orange,
      );
    case 3:
      return Icon(
        Icons.sentiment_neutral,
        size: 48,
        color: Colors.blue,
      );
    case 4:
      return Icon(
        Icons.sentiment_satisfied,
        size: 48,
        color: Colors.deepPurpleAccent,
      );
    default:
      return Icon(
        Icons.sentiment_very_satisfied,
        size: 48,
        color: Colors.green,
      );
  }
}
```

10. Run the program and insert a few emotion journals.

11. Click the menu View -> Tools Windows -> App Inspection. Select your mobile device and your application package name from the list. You will be able to inspect the database created by this app as below:



TODO: Enhance the app by showing the selected emoji.